SauceDemo UI Automation Framework

A maintainable browser automation project for validating an end-to-end e-commerce journey, from login through order confirmation.

44	58	3	CI
Test executions passing locally	Scenarios in the source catalogue	Configurable browsers Chrome · Firefox · Edge	GitHub Actions headless Chrome

Framework design

Page Object Model with reusable test infrastructure

Tests describe customer behavior and verify outcomes. Page objects keep locators and page actions together, while shared base classes handle browser lifecycle, waits, and common interactions.

Test flow and supporting layers

Clear ownership · isolated responsibilities

TestNG tests
Scenario setup
Assertions
Data providers

›

Page objects
Login · Inventory
Product · Cart
Checkout · Complete

›

Shared base
BaseTest lifecycle
BasePage waits
Reusable actions

›

WebDriver
Browser setup
Thread-local driver
Headless or visible

Validated application: **SauceDemo** · https://www.saucedemo.com/

Design choices

Tests stay readable
Test classes hold scenario intent and assertions. Page classes expose focused actions and read-only state.

Browser setup is centralized
BaseTest creates and closes browsers, captures failure screenshots, and records test outcomes.

Waits follow application state
Explicit waits check page navigation, cart counts, and checkout transitions instead of fixed sleeps.

Data stays outside test logic
Public demo credentials and customer values are read from a classpath properties file.

Technology: Java 25 · Selenium WebDriver 4.52 · TestNG 7.11 · Maven · WebDriverManager · Log4j · ExtentReports

What the suite validates

Independent browser tests cover high-value customer flows and the supporting checks that keep them reliable.

Login and session

Valid and rejected credentials, required-field errors, locked-out account behavior, password masking, logout, refresh, and direct inventory access after logout.

Inventory and products

Product names, prices and images; product detail content; return navigation; alphabetical and price sorting; responsive viewport checks.

Cart behavior

Add and remove products, badge consistency, multiple products, item name/description/price/quantity, continue shopping, and repeated add/remove actions.

Checkout and orders

Required customer fields, special and long input values, overview line items, calculated subtotal/tax/total, cancellation, successful completion, and Back Home.

Navigation and keyboard checks

Side menu, All Items and About links, app-state reset, browser back/forward, keyboard tab traversal, and single- and multi-item end-to-end purchase flows.

Reporting and failure evidence

- Log4j console and rolling-file logs under **target/logs/**
- Extent report at **target/extent-reports/**
- Automatic failure screenshots under **target/screenshots/**
- TestNG and Surefire XML/text reports under **target/surefire-reports/**

GitHub Actions

Workflow runs on pushes and pull requests to **main**, and can be started manually.

Ubuntu runner · Temurin Java 25 · Chrome headless · Maven clean verify.

The workflow has passed; the local suite also passes with 44 executions.

Run the suite

```
mvn --batch-mode --no-transfer-progress clean verify -Dbrowser=chrome -Dheadless=true
```

Run visibly or choose another browser

```
mvn clean test -Dheadless=false
mvn clean test -Dbrowser=firefox
mvn clean test -Dbrowser=edge
```

Source code github.com/vinjavarapu/SeleniumTestAutomation ↗

Scope note: the 58-case PDF is the design catalogue; the current executable suite automates broad functional coverage across those areas, not every listed case. SauceDemo is a public practice site, so UI behavior may change over time.